

Actividad 9 — Space Invaders (disparos y enemigos)

Duración: 50 min · **Modalidad:** individual · **Entrega:** repo GitHub público

1. Objetivos

- Trabajar con **listas de objetos** que se crean y destruyen (balas, enemigos).
- Implementar el movimiento de una **oleada de enemigos**.
- Detectar colisiones entre balas y enemigos.
- Manejar puntaje y fin de partida.

2. Marco teórico

Space Invaders (1978) es uno de los juegos más influyentes de la historia. Una nave dispara contra una flota de alienígenas que se mueve en bloque: van hacia un lado, al tocar el borde **bajan un nivel** y cambian de dirección.

Las ideas centrales:

1. **Listas para todo lo que se repite:** balas y enemigos son listas de Rect. Cuando una bala choca con un enemigo, ambos se eliminan de sus listas.
2. **Recorrer una copia al borrar:** si borras elementos de una lista mientras la recorrés con for, saltás elementos. Por eso usamos `for b in balas[:]:` (una copia).
3. **Detección de borde de la oleada:** si algún enemigo toca el borde, toda la oleada cambia de dirección y baja.
4. **Ciclo de vida de una bala:** se crea al apretar espacio, avanza hacia arriba y se elimina cuando sale de la pantalla (así no acumulamos memoria).

3. Paso a paso

```
cd mis-videojuegos
mkdir clase_09_space_invaders
```

Escribí `clase_09_space_invaders/invaders.py`:

```
import sys
import pygame
```

```
pygame.init()
```

```
ANCHO, ALTO = 800, 600
pantalla = pygame.display.set_mode((ANCHO, ALTO))
reloj = pygame.time.Clock()
```

```
nave = pygame.Rect(ANCHO // 2 - 30, ALTO - 60, 60, 30)
```

```

balas = [] # Lista de Rect (balas de la nave)
enemigos = [pygame.Rect(x, y, 40, 30)
              for x in range(40, ANCHO - 40, 60)
              for y in range(30, 150, 40)]
dir_enemigos = 1 # 1 derecha, -1 izquierda
puntos = 0

ejecutando = True
while ejecutando:
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            ejecutando = False
        elif evento.type == pygame.KEYDOWN and evento.key == pygame.K_SPACE:
            CE:
                balas.append(pygame.Rect(nave.centerx - 2, nave.y, 4, 12))

# Movimiento de la nave
teclas = pygame.key.get_pressed()
if teclas[pygame.K_LEFT]:
    nave.x -= 6
if teclas[pygame.K_RIGHT]:
    nave.x += 6
nave.x = max(0, min(ANCHO - nave.width, nave.x))

# Mover balas y eliminarlas si salen
for b in balas[:]:
    b.y -= 8
    if b.bottom < 0:
        balas.remove(b)

# Mover la oleada en bloque
borde = any(e.right >= ANCHO or e.left <= 0 for e in enemigos)
if borde:
    dir_enemigos *= -1
    for e in enemigos:
        e.y += 20
for e in enemigos:
    e.x += 3 * dir_enemigos

# Colisiones bala-enemigo
for b in balas[:]:
    for e in enemigos[:]:
        if b.colliderect(e):
            balas.remove(b)
            enemigos.remove(e)
            puntos += 10
            break

# COMPLETÁ: si un enemigo toca la nave (e.colliderect(nave))

```

```
# o llega abajo (e.bottom >= ALTO) -> fin del juego
```

```
# Dibujar
```

```
pantalla.fill((10, 10, 20))
pygame.draw.rect(pantalla, (0, 255, 120), nave)
for b in balas:
    pygame.draw.rect(pantalla, (255, 255, 100), b)
for e in enemigos:
    pygame.draw.rect(pantalla, (255, 80, 80), e)
pygame.display.set_caption(f"Space Invaders - Puntos: {puntos}")
pygame.display.flip()
reloj.tick(60)
```

```
pygame.quit()
sys.exit()
```

4. Actividad de cierre (para el commit)

1. Completá el bloque COMPLETÁ (colisión con la nave y enemigos que llegan abajo).
2. Agregá **vidas** (3) y que la nave parpadee un segundo al ser tocada antes de perder la vida.
3. Cuando se elimina toda la oleada, mostrá "¡Ganaste!".
4. Commit y push:

```
git add .
git commit -m "Clase 09: space invaders con oleadas, vidas y puntaje"
git push
```

5. Usando IA en esta clase (prompts sugeridos)

Prompt 1: “¿Por qué recorro las balas con `for b in balas[:]` y no con `for b in balas`? Mostrame qué pasa si lo hago mal.”

Prompt 2: “Agregá enemigos que disparen balas hacia abajo. Mi código: [pegá el código].”

Prompt 3: “Hacé que la oleada acelere a medida que hay menos enemigos (la velocidad depende de `len(enemigos)`).”

Prompt 4: “Convertí los rectángulos en sprites con imágenes. Dame las URLs de sprites libres y mostrame cómo cargarlos con `pygame.image.load`.”

6. Desafíos para seguir practicando

- Enemigos de **distintos colores** que valen distinto puntaje.
- Un **bonus** que cruza la pantalla cada tanto.
- Sonido de disparo (`pygame.mixer.Sound` con un archivo `.wav`).
- Agregar **escudos** que se van desgastando con los disparos.

7. Rúbrica de evaluación

Criterio	Logrado	En proceso	No logrado
Oleada en bloque	Correcta	Se mueve pero mal	No se mueve
Disparos	Dispara y borra balas	Dispara pero no borra	No dispara
Colisiones	Destruyen enemigos	A veces	No
Vidas y fin de juego	Correctos	Parcial	Falta